Parallelizing BatteryML with DDP

Alfred Tang*
PhD Tutor Hub
(Dated: April 7, 2026)

This work offers a tutorial on how to parallelize a serial Pytorch code called BatteryML using DDP (Distributed Data Parallel) and to interpret its outputs.

I. INTRODUCTION

Batteries are becoming more and more prevalent in the society. The prices of electric vehicles and charging time are continuously dropping while the driving range is constantly going up. Now it is possible for an electric vehicle (EV) to drive 300 miles after only 10 minutes of charge. Electric airplanes are also beginning to get some traction. Unmanned aerial vehicles (UAV's) are grabbing headlines in the Ukraine and Russia war. Iran War broke out during the time of the writing of this paper. Sales of EV increases tremendously because of the fear of oil shortage. EV is seen as a way to cushion the economic impacts of geopolitical conflicts in the Middle East. Both military and e-commerce companies such as Amazon are actively looking into drone swarms. All electric vehicles (including hybrids) run on battery. Reliable, fast charging and high capacity batteries are critical for the current technological trend. The characterization of battery, which is one of the topics of this work, is an important part of battery research.

Industry has been investing heavily on battery research for decades. That effort is accelerating rapidly in recent years because of the intense competition among EV companies. EV engine is much simpler than its gasoline counterpart. The mechanics of an EV is made up of just an electric motor and a battery. The quality of the battery is key for the success of an EV model to stand out in the competition. A lot of battery research focuses on the materials of the electrolytes and electrodes. Solid state battery is thought to be the future. Beside physics and chemistry, cost and geopolitics are also important considerations. Regardless of the types of battery used, there is always at least some lithium in it. Currently Chile controls the largest reserve of lithium, followed by Australia and China. Lithium is expensive—costing USD 5,000-8,000 per ton. Therefore battery manufacturers want to use as little of it as possible.

Battery degradation is directly linked to its longevity. All materials age. A battery loses its capacity to hold charge after so many charging cycles. In order for EV's to compete with gasoline cars, the battery has to have high energy density and be cheap and long lasting. When a battery degrades, it does not hold charge and loses power. EV battery is expensive—ranging from USD 5,000-20,000 a copy. In some cases, battery replacement costs as much as buying a new EV. Therefore battery degradation study is an important component of battery research.

Traditional battery degradation research based on experiment and theory is very expensive, time consuming and labor intensive. A battery cell needs to be charged and discharged many cycles to measure its degradation. Even with an automated setup, it takes a minimum of a month to obtain any usable data. In typical situations, the length of time to build a set of data is at least a year. In addition to the time factor, the combination of materials and design parameters is infinite. It is why battery researchers are exploiting AI to speed up development. AI generally operates on big data. The battery industry currently generates a petabyte of data a year [1]. But much of it is proprietary. Open source data are rare and much smaller in size. A 2024 paper claimed that the most comprehensive set of open source degradation data of lithium-ion NMC/C-SiO battery consists of just 228 cells [2]. This example offers a glimpse of how small open source battery data are. Even though industry can afford to generate a lot of data experimentally, the process takes too long and costs too much money. With machine learning, industry can speed up the development of battery and save money. Although a lot of good work has been done in using machine learning to facilitate battery design, the field is still young. The main challenge is to apply machine learning to small data. The concept of "less is more" is emerging by necessity. Talks of synthetic data and highly abstract mathematical formalism to circumvent small data are trendy in these conversations. This work deviates from that trend and chooses to work with real data and use simple analysis. If the idea works, it will accelerate battery design. BatteryML is an example of applying AI to battery characterization. The principles outlined here can be applied to a broader class of problems than just battery research. BatteryML is a Pytorch code written to run on a single GPU. This work shows how to parallelize it using Distributed Data Parallel (DDP) which ships with Pytorch by default. Prior to this work, there was not a comprehensive tutorial on how to parallelize a serial Pytorch code using DDP. There are snippets and suggestions posted online but they are fragmented and often do not compile nor run. The codes presented in this work have been tested on an HPC system and are comprehensive. The tutorial is followed by a discussion of the outputs of the the parallel version of BatteryML.

* atang@phdtutorhub.com

II. WHAT IS BATTERYML?

BatteryML is a machine learning code for the study of battery degradation. The code is archived in GitHub [3] along with a companion paper [4]. BatteryML is a compilation of 14 previously published machine learning codes (Table I) and a reasonably large collection of open source battery degradation data (Table II). The data sets are described in Appendix A of Reference [4]. Most of them can be downloaded online directly from various websites. A page called `dataprepare.md` on Reference [3] lists the websites that host the data, except SNL, HNEI and UL_PUR which need to be requested individually by sending an email to `info@batteryarchive.org`.

TABLE I. BatteryML Models

Model	Framework
Discharge model	Scikit-learn
Dummy	Scikit-learn
Full model	Scikit-learn
Gaussian process regression (gpr)	Scikit-learn
Principal component regression (pcr)	Scikit-learn
Principle least squares regression (plsr)	Scikit-learn
Random forest (rf)	Scikit-learn
Ridge regression (ridge)	Scikit-learn
Variance model	Scikit-learn
XGBoost (xgb)	Scikit-learn
Convolutional neural network (cnn)	Neural network
Long short-term memory (lstm)	Neural network
Multilayer perception (mlp)	Neural network
Transformer	Neural network

TABLE II. BatteryML Data Sets

Data Source	Electrode Chemistry	Cell Count
CALCE	LCO/graphite	13
HNEI	NMC, LCO/graphite	14
HUST	LFP/graphite	77
MATR	LFP/graphite	180
RWTH	NMC/carbon	48
SNL	NCA, NMC, LFP/graphite	18
UL_PUR	NCA/graphite	10
	Total number of cells:	360

Lithium ion is a broad category that includes subcategories such as nickel manganese cobalt (NMC), lithium iron phosphate (LFP), lithium cobalt oxide (LCO) and lithium nickel-cobalt-aluminum oxide (NCA). Figure 1 is a visualization of the MATR training data sets. In order to respect the copyright of published figures, this work regenerates the figures in Reference [4] by using slightly different data sets. The purpose of Figure 1 is to illustrate the line shapes of normalized battery charging capacity versus the number of charging cycles for 42 cells.

In battery research, battery state of charge (SOC) is inferred from open circuit voltage (OCV). This information is included in the data sets for each cell. The voltage of a battery cell rises as it charges and drops as it discharges. Figure 2 plots the voltage of a particular battery cell as a function of time as it charges and discharges through 49 cycles. Figures 1 and 2 are examples of the kinds of information that are stored in each data set.

FIG. 1. Typical MATR training data sets

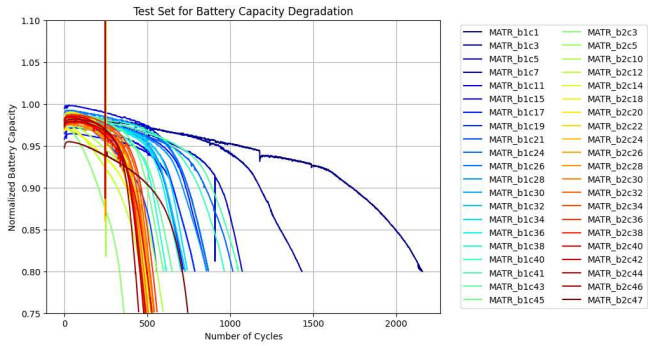
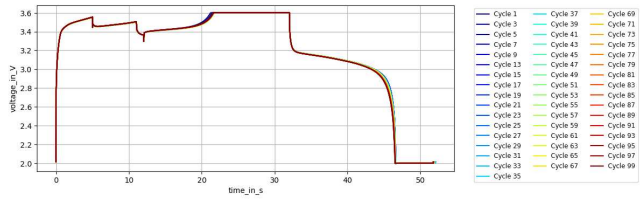


FIG. 2. Voltages of a Battery Cell in Charging and Discharging Cycles



To demonstrate the effectiveness of machine learning in battery research, the Scikit-learn variance model trained on the MATR data sets is used as an example. Modeling after a similar plot used in Reference [3], prediction is plotted against the ground truth in Figure 3. The ground truth refers to the correct answers. Ideally the prediction should be the same as the ground truth. In a perfect scenario, all the points should lie on the straight line. In this case, the points fall on both sides of the line. The agreement between prediction and the ground truth is acceptable within statistical errors.

BatteryML includes a couple of sample calculations in the form of jupyter notebooks that compute the root mean square errors (RMSE) between prediction and ground truth. Each data file contains descriptors, summary data and cycle data such as time, voltage, current, temperature, discharge capacity and so on. RMSE is calculated from all these quantities and can be used as a measure of the accuracy of training. Table III is the RMSE outputs of a sample calculation based on a BatteryML jupyter notebooks `baseline.ipynb` that runs on 1 GPU. By train_RMSE, it refers to RMSE of the train data set. There are two test sets labeled as 1 and 2. Hence test1_RMSE is RMSE of test set 1 and so on.

It is observed that the values of train_RMSE,

FIG. 3. Scikit-learn Prediction versus Ground Truth

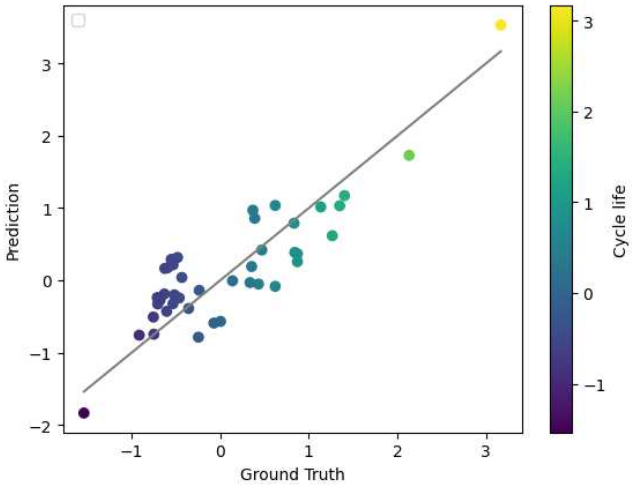


TABLE III. Root Mean Square Errors (RMSE) generated by `baseline.ipynb`

	Method	train_RMSE	test1_RSME	test2_RMSE
0	cnn	35.410439	81.305870	189.435364
1	lstm	41.803680	93.852249	251.073654
2	mlp	92.593361	99.231102	266.027283
3	transformer	41.137138	109.105713	312.723083
4	discharge_model	73.545250	114.216583	148.582092
5	dummy	362.026093	327.674103	510.628448
6	full_model	65.791832	126.043655	1074.400146
7	gpr	0.020461	87.694748	223.729150
8	pcr	88.151200	93.017391	197.508882
9	plsr	52.005884	90.336118	180.533886
10	rf	75.401234	184.251278	237.352354
11	ridge	47.797583	87.692564	183.799726
12	variance_model	116.000420	131.105087	211.236359
13	xgb	0.195780	237.191483	798.804321

test1_RMSE and test2_RMSE in Table III vary greatly even within the same models such as gpr and xgb. Small RMSE means that training and validation agree across all types of quantities such as discriptor, voltage, current, temperature, battery capacity and derived quantities. Large RMSE means the opposite. It is unreasonable that train RMSE and test RMSE are both small and large within the same model. One possible explanation is that a wrong model can occasionally gives the right answers. MATR is a very small data set. Like flipping a coin, it is possible to get 4 heads out of 4 tosses when one is lucky. But it does not mean that a single lucky experiment reflects the norm. The way to check what is really going on is to toss the coin a large number of times to obtain a more accurate distribution. Similarly the way to obtain reliable RMSE results in BatteryML is to increase the size of the data sets to check the scaling. A way to multiply a small data set is to generate many subsets by randomly sampling from the original data pool. The random samples are then fed to the same model in a data parallel framework. The purpose of this work is to test this idea.

Scikit-learn is considered "classical" machine learning that relies mostly on statistical methods with limited neural network capability. This work focuses on neural network exclusively so that only cnn, lstm, mlp and transformer models will be used. The goal of this study is not to design an inference model for battery design but is a proof of concept to show how to extend a small set of training data by parallelizing BatteryML using the data parallel model to improve the reliability of machine learning.

III. PARALLELIZING BATTERYML WITH DDP

In a data parallel model, all the processors are running the same model on different data. In this work, each rank ocuppies 1 GPU and operates on randomly selected subsets of the MATR data set. The training parameters and the outputs are averaged over the global ranks.

To deploy the data parallel model using DDP, 3 files in BatteryML need to be modified. They are `pipeline.py`, `nn_model.py` and `MATR_split.py`. The run script and launch script are user provided and not part of the BatteryML package. The run script `baseline.py` is extracted from the jupyter notebook `baseline.ipynb` that comes with BatteryML. The launch script `baseline.sh` is written for Polaris at ALCF. All 5 files that are needed to parallelize BatteryML using DDP is archived at GitHub [5]. The files are annotated with comments and are self-explanatory. This section provides some high level explanations of the code changes implemented in these files.

The first step of parallelization is to initialize DDP in `baseline.py`. It is good practice to define a cleanup function to release the memory when the code finishes. A barrier is called to wait for all the processes to finish before `dist.all_reduce` is invoked. BatteryML outputs RMSE as real numbers. DDP can only all reduce tensors so that all the RMSE values have to be converted from real numbers to tensors. In order to average RMSE over multiple ranks, it needs to be squared in each rank, averaged over all the ranks by `dist.all_reduce` with the `op=dist.ReduceOp.AVG` option, and then sqaure-rooted again at the end.

DDP uses the NCCL backend on Polaris because of its NVIDIA GPUs. There is no need to define the mapping between ranks and GPUs manually. However `baseline.sh` has to ssh to each node and launch `torchrun` on all the GPUs on that node.

In this work, only MATR data sets are used. There are 180 MATR data sets. One of them is an outlier and two are omitted inadvertently. The rest of 177 is split into 107 training sets and 70 testing sets. For each rank, 100 out of 107 training sets and 60 out of 70 testing sets

are randomly chosen.

$$\binom{107}{100} = 26075972546, \quad \binom{70}{60} = 396704524216. \quad (1)$$

Polaris has 560 nodes. Each node has 1 AMD EPYC "Milan" CPU and 4 NVIDIA A100 GPUs. As a policy, the `prod` queue can only use up to 496 nodes. This work deploys a maximum of 400 nodes. Each rank runs on 1 GPU. Therefore the maximum number of ranks is 1600. As it is shown in Eq. 1, there are more than enough unique combinations among the random subsets to populate all the ranks. The split between training and testing data is handled by the file `MATR_split.py`. The random seed is set to the global rank. The selection of data ID's is handled by the function `random.sample`. This way, each rank operates on different data.

BatteryML writes the data to cache after the data is loaded the first time. The parallelized version of BatteryML operates on different data for each rank and cannot load the data from the cache directory. Therefore it is important that the cache directory is emptied and the lines for writing to cache in `pipeline.py` are disabled.

Global ranks go over all the ranks while local ranks range from 0 to 3 for the 4 GPUs in each node. It is important that local rank is used for `torch.cuda.set_device` and global rank is used for `DistributedDataParallel` in `nn_model.py`. As usual, a barrier is needed in the right place to prevent a racing condition.

IV. RESULTS AND DISCUSSION

The parallel version of BatteryML runs on 1, 2, 10, 20, 40, 60, 80, 100, 200, 300 and 400 nodes of Polaris at ALCF. The number of ranks is the number of nodes times 4. RMSE versus total number of ranks for the 4 neural network models are plotted in Figures 4-7.

Figure 4 is for the cnn model. It is observed that RMSE fluctuates until the number of ranks reaches 400. This observation applies to Figures 5-7 as well. The train data test is larger than the test data set. Yet RMSE of the train set is larger than those of the test sets, which is the opposite of what is expected. Additionally RMSE of the train and test sets do not follow the same trend exactly even though the data sets are very similar, which is a feature that is different from the other 3 figures.

Unlike the cnn model, RMSE of the train and test sets follow the same trend in Figure 5 of the lstm model. What is not expected is the large difference between RMSE of two test sets relative to the train data set because the two test sets are trained by the same number of data sets and should have similar RMSE.

In Figure 6 of the mlp model, RMSE of the train set and test set 1 follow the same trend but test set 2 does not. RMSE of test set 2 is also too close to that of the train set. These features are unexpected.

FIG. 4. RMSE of cnn

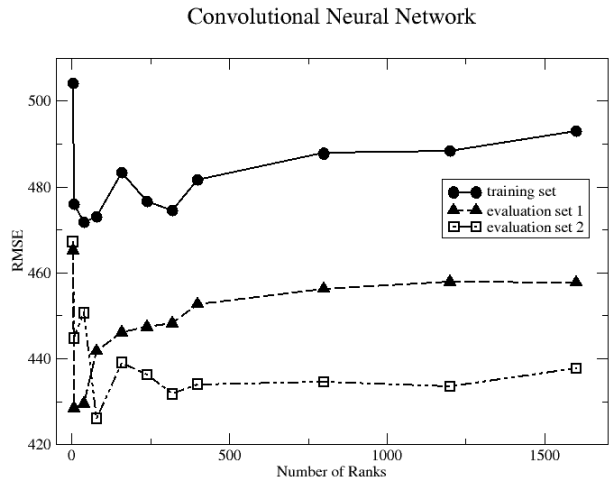
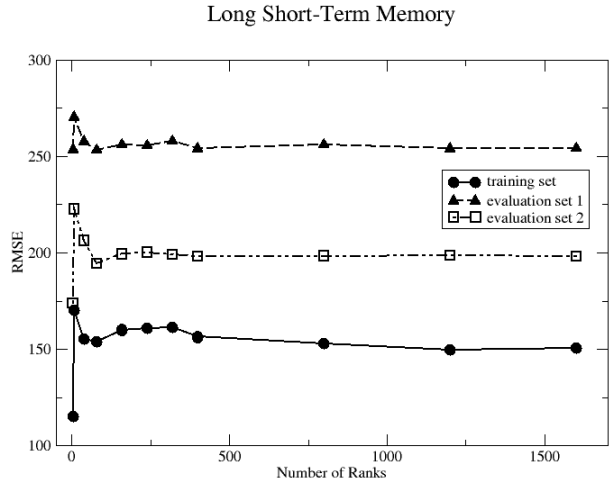


FIG. 5. RMSE of lstm



Finally, Figure 7 of the transformer model is the most consistent among all the models. RMSE of test sets 1 and 2 are very close to each other and are larger than that of the train set. RMSE of all 3 data sets follow the same trend beyond 400 ranks. All these features are expected.

It is often said that neural network is a black box. The behaviors of a model are not always predictable. Hyperparameters need to be tuned by trial and error to optimize a model. As it is aforementioned, this work does not aim to optimize any specific machine learning models but is a proof of concept to show how to parallelize a serial Pytorch code and to glean new information from the parallelized code. There are several observations that can be drawn from Figures 4-7: (1) The outputs of all 4 mod-

FIG. 6. RMSE of mlp

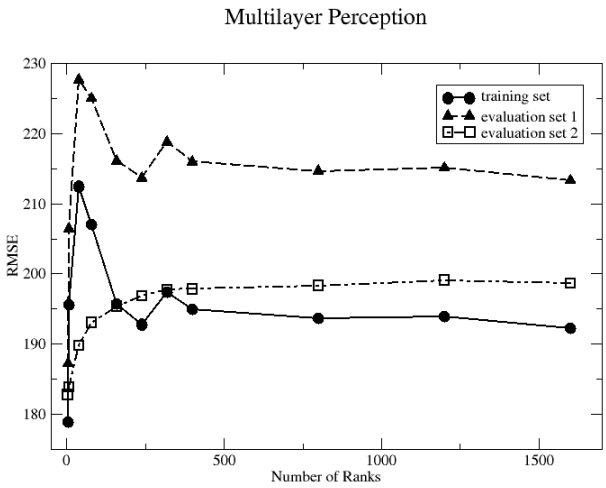
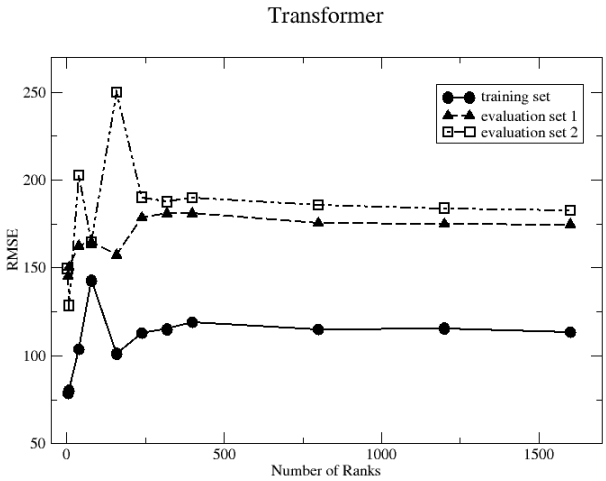


FIG. 7. RMSE of transformer



els are consistent–namely there are always fluctuations in RMSE prior to rank 400 and the trend lines are mostly flat after that. Consistency is a sign that the parallelization of BatteryML works correctly. (2) The fluctuations of RMSE among train and test sets do not always exhibit the same trends. They are considered transient properties. (3) RMSE of test sets are sometimes smaller than that of the train set even though test sets have less data than the train set, which is a sign that something is not right. The behaviors of neural network models are sensitive to the choice of hyper-parameters. Since optimization is not an objective of this work, these open questions will hopefully stir the interest of future projects. The hypothesis is that all the models can be optimized further to have similar features as the transformer model by tuning the hyper-parameters. Furthermore, the sizes of the train and test sets can be adjusted according to the selection criteria in Equations 1. All these improvements can be achieved in future works.

V. CONCLUSION

The DDP version of BatteryML is shown to work correctly. This work is a proof of concept. Hyper-parameters and the sizes of the train and test sets can be fine-tuned in future works to improve the models. It will be interesting to see if RMSE of all the other models can be made to follow similar patterns as those in the transformer model after fine-tuning. The DDP version of BatteryML runs on Polaris which has NVIDIA GPUs. Future works can extend the parallelization techniques to AMD and Intel GPUs by using the RCCL and oneCCL backends respectively. The techniques of parallelizing Pytorch using DDP as shown in this work can be applied to all other relevant machine learning models in more general contexts.

ACKNOWLEDGMENTS

This work is supported by the Director's Discretion Allocation Program of ALCF at Argonne National Lab, USA.

[1] Teo Lombardo, *et el.*, *Chemical Reviews*, **122**, 10899–10969 (2022).
[2] Matthia Luh and Thomas Blank, "Comprehensive battery aging dataset: capacity and impedance fade measurements of a lithium-ion NMC/C-SiO cell", *Nature Science Data*, **11**, 1004 (2024).
[3] https://github.com/microsoft/BatteryML
[4] Han Zhang, Xiaofan Gui, Shun Zheng, Ziheng Lu, Yuqi Li and Jiang Bian, "BatteryML: An Open-source Platform for Machine Learning on Battery Degradation", *The Twelfth International Conference on Learning Representations* (2024).
[5] https://github.com/phdtutorhub/Parallelizing-BatteryML-with-DDP